

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT on

DATA STRUCTURES USING C(23CS3PCDST)

Submitted by

Ghanshyam Sharma (1BM23CS100)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “**DATA STRUCTURES USING C(23CS3PCDST)**” carried out by **Ghanshyam Sharma (1BM23CS100)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2022. The Lab report has been approved as it satisfies the academic requirements in respect of a **DATA STRUCTURES USING C(23CS3PCDST)** work prescribed for the said degree.

Namratha M Assistant Professor Department of CSE, BMSCE	Dr. Joythi S Nayak Professor & HOD Department of CSE, BMSCE
---	---

Index

Sl. No.	Date	Experiment Title	Page No.
1	3-10-2024	Stacks	4-7
2	10-10-2024	infix to postfix leetcode account creation	8-9
3	24-10-2024	leetcode queue and circular queue	10-20
4	7-11-2024	singly linked list insertion and display leetcode	21-26
5	14-11-2024	singly linked list deletion and display leetcode	27-35
6	21-11-2024	sort,reverse,concatenation of linked list stack and queue using linked list	36-45
7	28-11-2024	doubly linked list	45-50
8	19-12-2024	binary search tree traversal and display	50-55
9	4-12-2024	bfs method traversal to check graph is connected or not using dfs	55-59
10	4-12-2024	hashing	59-62

LAB 1 STACKS

Write a program to simulate the working of stack using an array with the following:

a) Push

b) Pop

c) Display

The program should print appropriate messages for stack overflow, stack underflow.

```
#include<stdio.h>
#include<stdlib.h>
#define SIZE 5
int top=-1;
int stack[SIZE];
int isfull(){
    return top==SIZE-1?1:0;
}
int isempty(){
    return (top==-1)?1:0;
}
void push(int i){
    if(isfull()){
        printf("stack overflow\n");
        return;
    }
    stack[++top]=i;
}
int pop(){
    if(isempty()){
        printf("stack underflow\n");
        return 0;
    }
    return stack[top--];
}
void display(){
    for(int i=top;i>=0;i--){
        printf("%d ",stack[i]);
    }
    printf("\n");
}
void main(){
    while(1){
        int ch;
        int i;
        printf("main menu \n1.push() 2.pop() 3.display() 4.isempty() 5.isfull() 6.exit\n");
        scanf("%d",&ch);
```

```

switch(ch){
    case 1:
        printf("\nenter the number\n");
        scanf("%d",&i);
        push(i);
        break;
    case 2:
        printf("the value popped is %d",pop());
        break;
    case 3:
        printf("the stack elements are as follows\n");
        display();
        break;
    case 4:
        if(isempty()){
            printf("empty\n");

        }else{
            printf("not empty\n");
        }
        break;
    case 5:
        if(isfull()){
            printf("full\n");

        }else{
            printf("not full\n");
        }
        break;
    case 6:
        exit(0);
        break;
}
}
}
output:

```

```
main menu
1.push() 2.pop() 3.display() 4.isempty() 5.isfull() 6.exit
1

enter the number
10
main menu
1.push() 2.pop() 3.display() 4.isempty() 5.isfull() 6.exit
1

enter the number
11
main menu
1.push() 2.pop() 3.display() 4.isempty() 5.isfull() 6.exit
3
the stack elements are as follows
11 10
main menu
1.push() 2.pop() 3.display() 4.isempty() 5.isfull() 6.exit
2
the value popped is 11main menu
1.push() 2.pop() 3.display() 4.isempty() 5.isfull() 6.exit
3
the stack elements are as follows
10
main menu
1.push() 2.pop() 3.display() 4.isempty() 5.isfull() 6.exit
2
the value popped is 10main menu
1.push() 2.pop() 3.display() 4.isempty() 5.isfull() 6.exit
3
the stack elements are as follows
```

the stack elements are as follows

main menu

1.push() 2.pop() 3.display() 4.isempty() 5.isfull() 6.exit

4

empty

main menu

1.push() 2.pop() 3.display() 4.isempty() 5.isfull() 6.exit

5

not full

main menu

1.push() 2.pop() 3.display() 4.isempty() 5.isfull() 6.exit

6

=== Code Execution Successful ===

Lab program 2:

Write a program to convert Infix to Postfix expression.

// Online C compiler to run C program online

```
#include <stdio.h>
#include <ctype.h>
#define MAX 50
int top=-1;
char stack[MAX];
void push(char a){
    stack[++top]=a;
}
char pop(){
    return stack[top--];
}
int pr(char a){
    if(a=='*' || a=='/')
        return 2;
    if(a=='+' || a=='-')
        return 1;
    return 0;
}
int main() {
    char infix[MAX],postfix[MAX];
    int i=0,k=0;
    printf("enter the infix expression\n");
    scanf("%s",infix);
    push('#');
    char ch;
    while((ch=infix[i++])!='\0'){
        if(ch=='(')
            push(ch);
        else if(isalnum(ch))
            postfix[k++]=ch;
        else if(ch==')'){
            while(stack[top]!='(')
                postfix[k++]=pop();
            pop();
        } else {
            while (stack[top]!='#' && pr(stack[top])>=pr(ch))
                postfix[k++]=pop();
            push(ch);
        }
    }
    while (stack[top]!='#')
```



```
        postfix[k++]=pop();
    postfix[k]='\0';
    printf("the postfix expression is %s",postfix);
    return 0;
}
```

output:

```
enter the infix expression
2+3+(6+7)*8
the postfix expression is 23+67+8*+
=== Code Execution Successful ===
```

Lab program 3:

WAP to simulate the working of a queue of integers using an array. Provide the following operations: Insert, Delete, Display

The program should print appropriate messages for queue empty and queue overflow conditions.

linear queue:

// Online C compiler to run C program online

```
#include <stdio.h>
```

```
#define MAX 50
```

```
int front=-1;
```

```
int rear=-1;
```

```
int queue[MAX];
```

```
int isEmpty(){
```

```
    if(front==0){
```

```
        return 1;
```

```
    }
```

```
    return 0;
```

```
}
```

```
int isFull(){
```

```
    if(rear==MAX-1)
```

```
        return 1;
```

```
    return 0;
```

```
}
```

```
void enqueue(int val){
```

```
    if(isFull()){
```

```
        printf("queue overflow\n");
```

```
        return;
```

```
    }
```

```
    if(front==0){
```

```
        front=0;
```

```
    }
```

```
    queue[++rear]=val;
```

```
}
```

```
int dequeue(){
```

```
    if(isEmpty()){
```

```
        printf("queue underflow\n");
```

```
        return -1;
```

```
    }
```

```
    int el=queue[front];
```

```
    if(front==rear){
```

```
        front=rear=-1;
```

```
        return el;
```

```

    }
    front++;
    return el;
}
void display(){
    if(isEmpty()){
        printf("empty queue\n");
        return;
    }
    printf("\n the elements of the queue are\n");
    for(int i=front;i<=rear;i++){
        printf("%d ",queue[i]);
    }
    printf("\n");
}
void main(){
    int ch;
    int val;
    do{
        printf("the mainmenu is as follows\n");
        printf("1.enqueue\n2.dequeue\n3.isFull\n4.isEmpty\n");
        scanf("%d",&ch);
        switch(ch){
            case 1:
                printf("enter the value to insert\n");
                scanf("%d",&val);
                enqueue(val);
                display();
                break;
            case 2:
                dequeue();
                display();
                break;
            case 3:
                printf("the queue is %s",isFull()?"full\n":"not full\n");
                break;
            case 4:
                printf("the queue is %s",isEmpty()?"empty\n":"not empty\n");
                break;
        }
    }while (ch!=5);
}

```

```
the mainmenu is as follows
1.enqueue
2.dequeue
3.isFull
4.isEmpty
1
enter the value to insert
10

the elements of the queue are
10
the mainmenu is as follows
1.enqueue
2.dequeue
3.isFull
4.isEmpty
1
enter the value to insert
11

the elements of the queue are
10 11
the mainmenu is as follows
1.enqueue
2.dequeue
3.isFull
4.isEmpty
```

2

the elements of the queue are

11

the mainmenu is as follows

1.enqueue

2.dequeue

3.isFull

4.isEmpty

2

empty queue

the mainmenu is as follows

1.enqueue

2.dequeue

3.isFull

4.isEmpty

3

the queue is not full

the mainmenu is as follows

1.enqueue

2.dequeue

3.isFull

4.isEmpty

4

the queue is empty

the mainmenu is as follows

```
4
the queue is empty
the mainmenu is as follows
1.enqueue
2.dequeue
3.isFull
4.isEmpty
5
```

circular queue:

// Online C compiler to run C program online

```
#include <stdio.h>
#define MAX 50
int front=-1;
int rear=-1;
int queue[MAX];
int isEmpty(){
    if(front==1){
        return 1;
    }
    return 0;
}
int isFull(){
    if((rear+1)%MAX==front)
        return 1;
    return 0;
}
void enqueue(int val){
    if(isFull()){
        printf("queue overflow\n");
        return;
    }
    if(front==1){
        front=0;
    }
    rear=(rear+1)%MAX;
    queue[rear]=val;
}
int dequeue(){
```

```

if(isEmpty()){
    printf("queue underflow\n");
    return -1;
}
int el=queue[front];
if(front==rear){
    front=rear=-1;
    return el;
}
front=(front+1)%MAX;
return el;
}
void display(){
    if(isEmpty()){
        printf("empty queue\n");
        return ;
    }
    printf("\n the elements of the queue are\n");
    int i;
    for( i=front;i!=rear;i=(i+1)%MAX){
        printf("%d ",queue[i]);
    }
    printf("%d",queue[i]);
    printf("\n");
}
void main(){
    int ch;
    int val;
    do{
        printf("the mainmenu is as follows\n");
        printf("1.enqueue\n2.dequeue\n3.isFull\n4.isEmpty\n");
        scanf("%d",&ch);
        switch(ch){
            case 1:
                printf("enter the value to insert\n");
                scanf("%d",&val);
                enqueue(val);
                display();
                break;
            case 2:
                dequeue();
                display();

```

```
        break;
    case 3:
        printf("the queue is %s",isFull()?"full\n":"not full\n");
        break;
    case 4:
        printf("the queue is %s",isEmpty()?"empty\n":"not empty\n");
        break;
    }
} while (ch!=5);
}
```

output:

the mainmenu is as follows

1.enqueue

2.dequeue

press and hold

Esc

4.isEmpty

1

enter the value to insert

10

the elements of the queue are

10

the mainmenu is as follows

1.enqueue

2.dequeue

3.isFull

4.isEmpty

1

enter the value to insert

12

the elements of the queue are

10 12

the mainmenu is as follows

1.enqueue

2.dequeue

3.isFull

4.isEmpty

2

the elements of the queue are

12

the mainmenu is as follows

1.enqueue

```
12
the mainmenu is as follows
1.enqueue
2.dequeue
3.isFull
4.isEmpty
2
empty queue
the mainmenu is as follows
1.enqueue
2.dequeue
3.isFull
4.isEmpty
4
the queue is empty
the mainmenu is as follows
1.enqueue
2.dequeue
3.isFull
4.isEmpty
2
queue underflow
empty queue
the mainmenu is as follows
1.enqueue
2.dequeue
3.isFull
```

```
3.isFull
4.isEmpty
5
```

```
=== Code Exited With Errors ===
```

LEETCODE:

```
typedef struct {
    int data[100000];
    int top;
}stack;

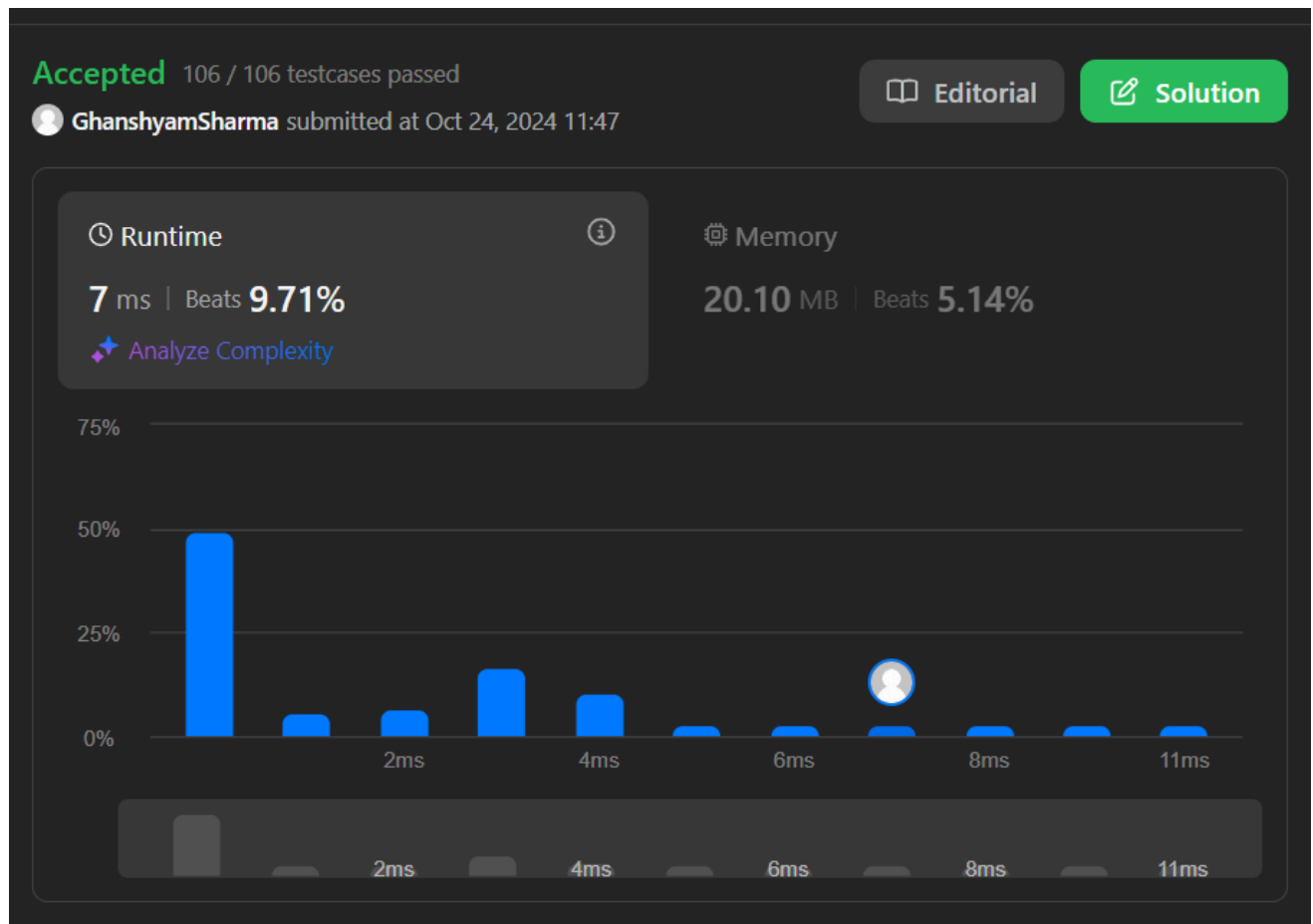
void push(char a,stack *st){
    st->data[++(st->top)]=a;
}
char pop(stack *st){
    return st->data[st->top--];
}
char top(stack *st){
    return st->data[st->top];
}
int isEmpty(stack *st){
    return st->top==-1;
}
void string(stack *st,char *ans){
    int i;
    for(i=0;i<=st->top;i++){
        ans[i]=st->data[i];
    }
    ans[i]='\0';
}

char* removeDuplicates(char* s) {
    stack *st =(stack*) malloc(sizeof(stack));
    st->top=-1;
    for(int i=0;s[i]!='\0';i++){
```

```

    if(!isEmpty(st) && s[i]==top(st)){
        pop(st);
    }else{
        push(s[i],st);
    }
}
char *ans =(char *)malloc(sizeof(char)*100000);
string(st,ans);
return ans;
}

```



Lab program 4:

WAP to Implement Singly Linked List with following operations

a) Create a linkedlist.

b) Insertion of a node at first position, at any position and at end of list.

Display the contents of the linked list.

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct node {  
    int data;  
    struct node *next;  
};  
typedef struct node *n;
```

```
void insertAtHead(n *head, int data) {  
    n temp = (n)malloc(sizeof(struct node));  
    temp->data = data;  
    temp->next = *head;  
    *head = temp;  
}
```

```
void insertAtTail(n *head, int data) {  
    n temp = (n)malloc(sizeof(struct node)), t = *head;  
    temp->data = data;  
    temp->next = NULL;  
    if (*head == NULL) {  
        *head = temp;  
        return;  
    }  
    while (t->next) {  
        t = t->next;  
    }  
    t->next = temp;  
}
```

```
void insertAtPos(n *head, int data, int pos) {  
    n temp = (n)malloc(sizeof(struct node)), t = *head;
```

```

temp->data = data;
if (pos == 0 || *head == NULL) {
    insertAtHead(head, data);
    return;
}
while (--pos && t != NULL) {
    t = t->next;
}
if (t == NULL) {
    printf("The index you gave is invalid\n");
    return;
}
temp->next = t->next;
t->next = temp;
}

```

```

void display(n *head) {
    n t = *head;
    if (*head == NULL) {
        printf("Empty linked list\n");
        return;
    }
    printf("\nThe linked list is as follows:\n");
    while (t) {
        printf("%d ", t->data);
        t = t->next;
    }
    printf("\n");
}

```

```

void main() {
    int ch;
    n head = NULL;
    while (1) {
        int val, pos;
        printf("\nEnter the option\n1. Insert at head 2. Insert at tail 3. Insert at position 4.
Display\n");
        scanf("%d", &ch);
    }
}

```

```

switch (ch) {
case 1:
    printf("Enter the value:\n");
    scanf("%d", &val);
    insertAtHead(&head, val);
    break;
case 2:
    printf("Enter the value:\n");
    scanf("%d", &val);
    insertAtTail(&head, val);
    break;
case 3:
    printf("Enter the value and position:\n");
    scanf("%d %d", &val, &pos);
    insertAtPos(&head, val, pos);
    break;
case 4:
    display(&head);
    break;
default:
    exit(0);
}
}
}
output:

```

```
Enter the option
1.head 2.tail 3.pos 4. Display
1
Enter the value:
10

Enter the option
1.head 2.tail 3.pos 4. Display
1
Enter the value:
11

Enter the option
1.head 2.tail 3.pos 4. Display
4

The linked list is as follows:
11 10

Enter the option
1.head 2.tail 3.pos 4. Display
2
Enter the value:
13

Enter the option
1.head 2.tail 3.pos 4. Display
4

The linked list is as follows:
11 10 13

Enter the option
1.head 2.tail 3.pos 4. Display
3
Enter the value and position:
23 2

Enter the option
1.head 2.tail 3.pos 4. Display
4

The linked list is as follows:
11 10 23 13

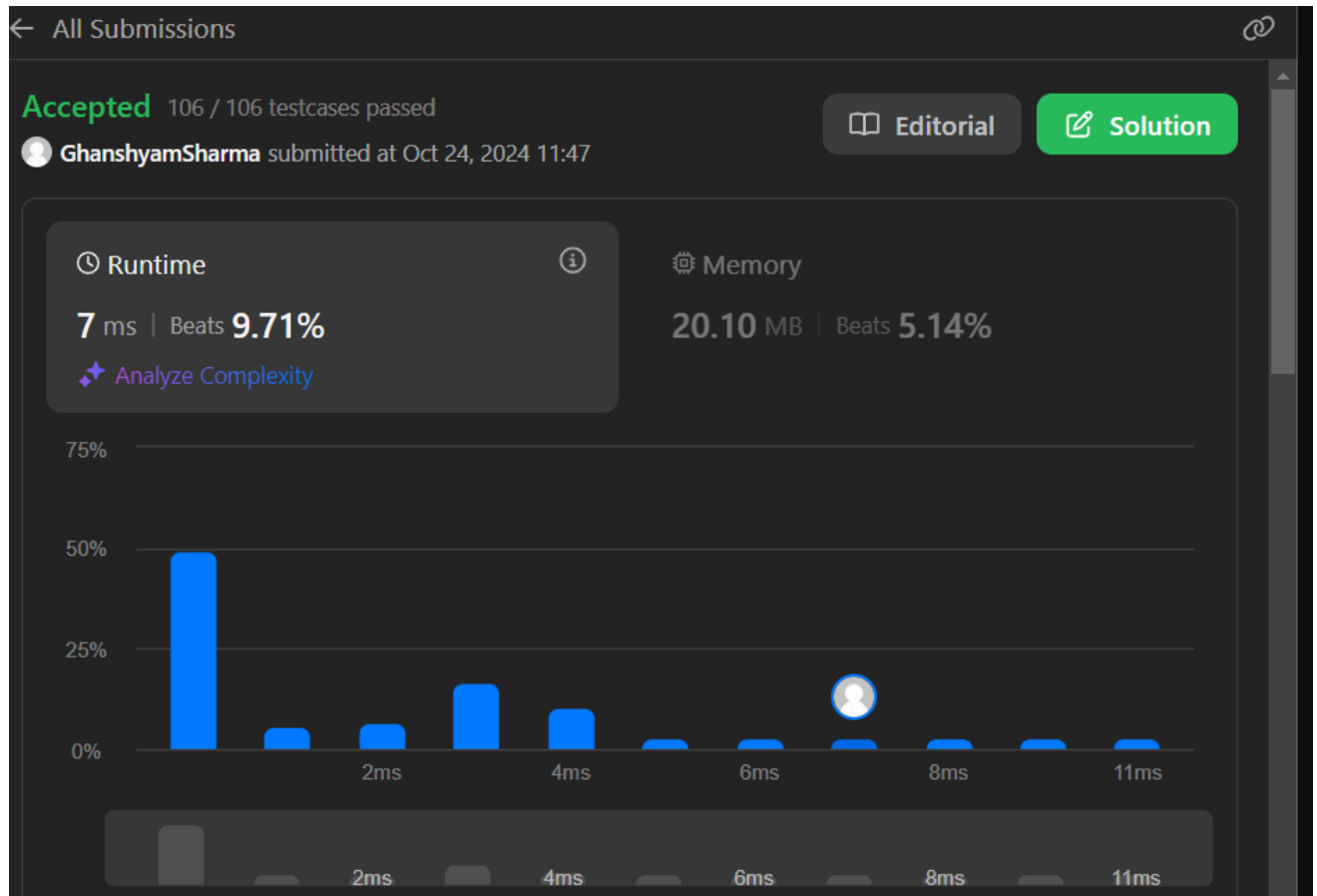
Enter the option
1.head 2.tail 3.pos 4. Display
5
```


LEETCODE 844:

```
typedef struct {
    int data[100000];
    int top;
}stack;

void push(char a,stack *st){
    st->data[++(st->top)]=a;
}
char pop(stack *st){
    return st->data[st->top--];
}
char top(stack *st){
    return st->data[st->top];
}
int isEmpty(stack *st){
    return st->top==-1;
}
void string(stack *st,char *ans){
    int i;
    for(i=0;i<=st->top;i++){
        ans[i]=st->data[i];
    }
    ans[i]='\0';
}
char* removeDuplicates(char* s) {
    stack *st =(stack*) malloc(sizeof(stack));
    st->top=-1;
    for(int i=0;s[i]!='\0';i++){
        if(!isEmpty(st) && s[i]==top(st)){
            pop(st);
        }else{
            push(s[i],st);
        }
    }
    char *ans =(char *)malloc(sizeof(char)*100000);
    string(st,ans);
}
```

```
return ans;  
}
```



Lab program 5:

WAP to Implement Singly Linked List with following operations

a) Create a linkedlist.

b) deletion of a node at first position, at any position and at end of list.

Display the contents of the linked list.

PROGRAM:

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct node {  
    int data;  
    struct node *next;
```

```
};
```

```
typedef struct node *n;
```

```
void insertAtHead(n *head, int data) {  
    n temp = (n)malloc(sizeof(struct node));  
    temp->data = data;  
    temp->next = *head;  
    *head = temp;  
}
```

```
void deleteAtHead(n *head) {  
    if (*head == NULL) {  
        printf("Can't delete from an empty linked list\n");  
        return;  
    }  
    n temp = *head;  
    *head = (*head)->next;  
    free(temp);  
}
```

```
void deleteAtTail(n *head) {  
    if (*head == NULL) {  
        printf("Can't delete from an empty linked list\n");  
        return;  
    }  
    if ((*head)->next == NULL) {  
        free(*head);  
    }
```

```

        *head = NULL;
        return;
    }
    n t = *head;
    while (t->next->next) {
        t = t->next;
    }
    free(t->next);
    t->next = NULL;
}

void deleteAtPos(n *head, int pos) {
    if (*head == NULL) {
        printf("Can't delete from an empty linked list\n");
        return;
    }
    if (pos == 0) {
        deleteAtHead(head);
        return;
    }
    n t = *head;
    while (--pos && t->next != NULL) {
        t = t->next;
    }
    if (t->next == NULL) {
        printf("The index you gave is invalid\n");
        return;
    }
    n temp = t->next;
    t->next = temp->next;
    free(temp);
}

void display(n *head) {
    n t = *head;
    if (*head == NULL) {
        printf("Empty linked list\n");
        return;
    }
    printf("\nThe linked list is as follows:\n");
    while (t) {

```

```

        printf("%d ", t->data);
        t = t->next;
    }
    printf("\n");
}

void main() {
    int ch;
    n head = NULL;
    while (1) {
        int val, pos;
        printf("\nEnter the option\n1. Add 2. Delete from head 3. Delete from end 4. Delete
from position 5. Display\n");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
                printf("Enter the value:\n");
                scanf("%d", &val);
                insertAtHead(&head, val);
                break;
            case 2:
                deleteAtHead(&head);
                break;
            case 3:
                deleteAtTail(&head);
                break;
            case 4:
                printf("Enter the position:\n");
                scanf("%d", &pos);
                deleteAtPos(&head, pos);
                break;
            case 5:
                display(&head);
                break;
            default:
                exit(0);
        }
    }
}

```

```
Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
1
Enter the value:
12

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
1
Enter the value:
13

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
1
Enter the value:
14

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
1
Enter the value:
15

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
5

The linked list is as follows:
15 14 13 12

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
4
Enter the position:
2

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
5

The linked list is as follows:
15 14 12

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
2

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
5
```

```

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
5

The linked list is as follows:
14 12

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
3

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
5

The linked list is as follows:
14

Enter the option
1. Add 2. Delete from head 3. Delete from end 4. Delete from position 5. Display
6

```

leetcode 83

```

struct ListNode* deleteDuplicates(struct ListNode* head) {
    typedef struct ListNode *node;
    node prev=NULL;
    node curr=head;
    while(curr){
        if(prev && prev->val==curr->val){
            node temp=curr;
            prev->next=temp->next;
            curr=prev->next;
        }else{
            node temp=curr;
            curr=curr->next;
            prev=temp;
        }
    }
    return head;
}

```

Accepted 168 / 168 testcases passed

GhanshyamSharma submitted at Nov 14, 2024 11:54

Editorial

Solution

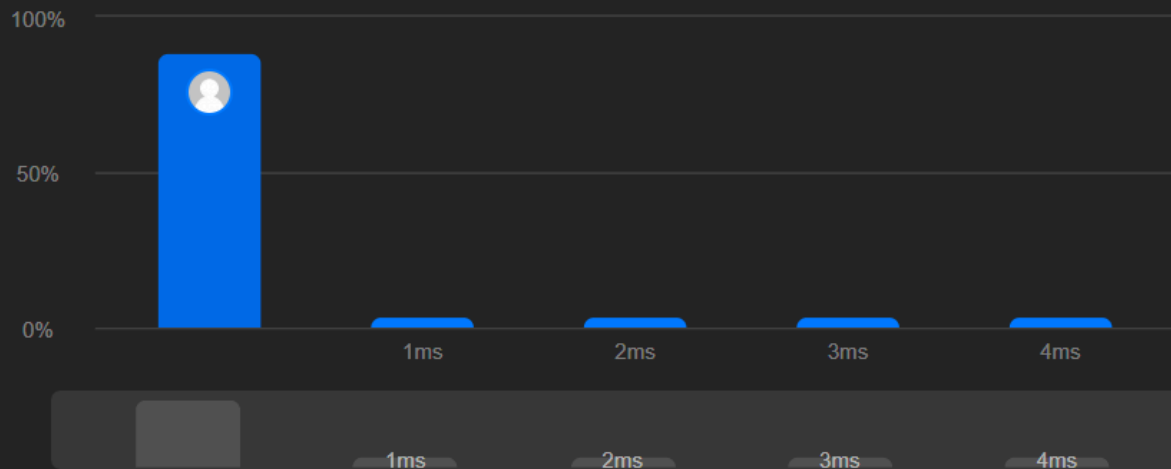
Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

10.98 MB | Beats 55.17%



leetcode 141

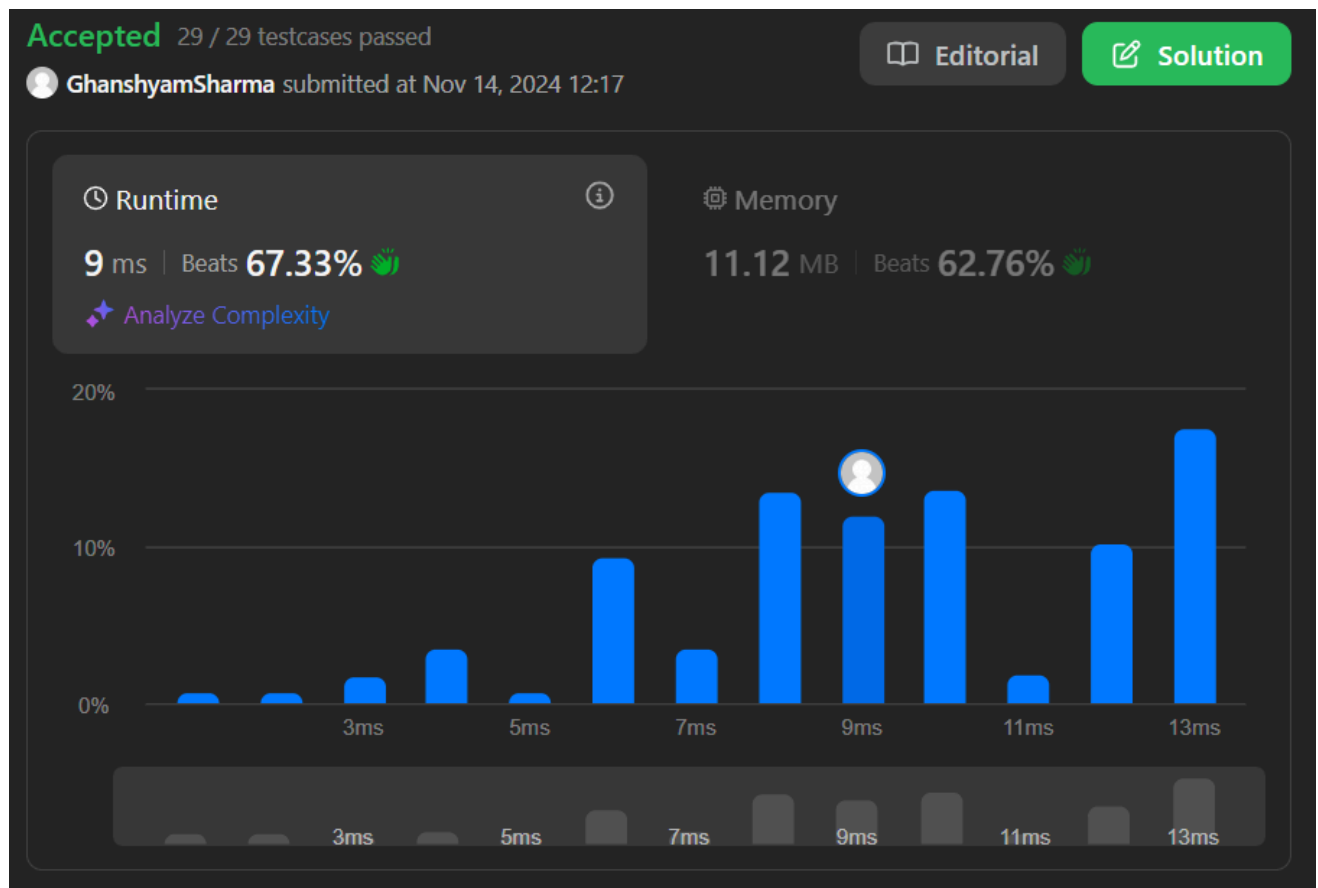
```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */
bool hasCycle(struct ListNode *head) {
    typedef struct ListNode * node;
    if(!head){
        return false;
    }
    if(head->next==head){
        return true;
    }else if(head->next==NULL){
        return false;
    }
}
```



```

node fast=head;
node slow=head;
while(fast){
    if(!fast->next){
        return false;
    }
    fast=fast->next->next;
    slow=slow->next;
    if(fast==slow){
        return true;
    }
}
return false;
}

```



leetcode 234

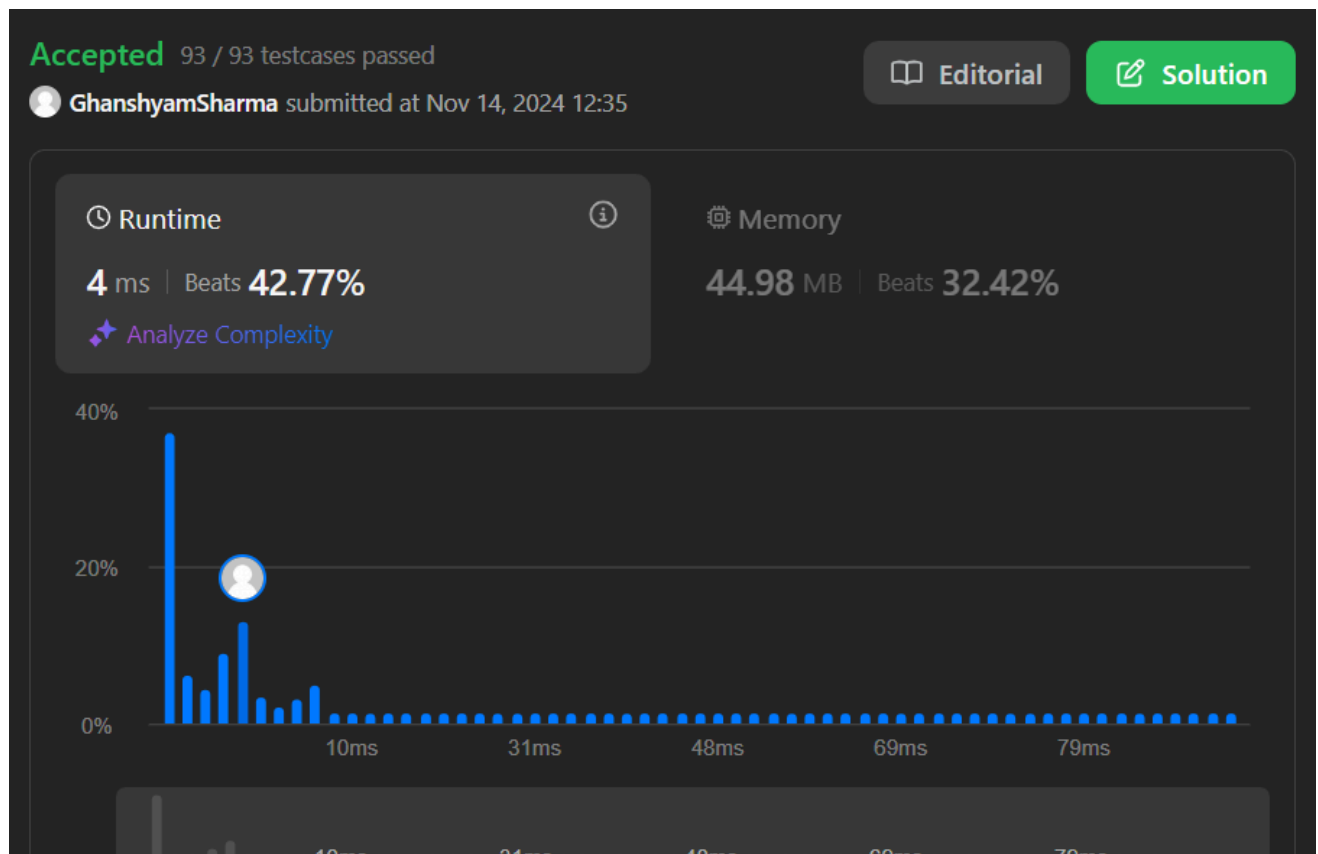
/**

* Definition for singly-linked list.

```

* struct ListNode {
*     int val;
*     struct ListNode *next;
* };
*/
typedef struct ListNode *node;
node reverse(node head){
    if(head==NULL || head->next==NULL){
        return head;
    }
    node prev=NULL;
    node curr=head;
    while(curr){
        node temp=curr->next;
        curr->next=prev;
        prev=curr;
        curr=temp;
    }
    return prev;
}
bool isPalindrome(struct ListNode* head) {
    int arr[100000];
    int size=0;
    node temp=head;
    while(temp){
        arr[size]=temp->val;
        temp=temp->next;
        size++;
    }
    node rev=reverse(head);
    for(int i=0;i<size;i++){
        if(!(arr[i]==rev->val)){
            return false;
        }
        rev=rev->next;
    }
    return true;
}

```



LAB PROGRAM 6:

- 1.SORT ,REVERSE AND CONCATENATE
- 2.STACK AND QUEUE USING LINKED LIST

1.sort

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct n {  
    int data;  
    struct n *next;
```

```
};
```

```
typedef struct n *node;
```

```
node insertAtHead(int data, node head) {  
    node temp = (node)malloc(sizeof(struct n));  
    temp->data = data;  
    temp->next = NULL;  
    if (head == NULL)  
        return temp;
```

```

    temp->next = head;
    head = temp;
    return head;
}

node bubbleSort(node head) {
    node end = NULL;
    int swap = 0;
    while (1) {
        node curr;
        swap = 0;
        for (curr = head; curr->next != end; curr = curr->next) {
            if (curr->data > curr->next->data) {
                int temp = curr->data;
                curr->data = curr->next->data;
                curr->next->data = temp;
                swap = 1;
            }
        }
        end = curr;
        if (!swap)
            break;
    }
    return head;
}

void display(node head) {
    if (head == NULL) {
        printf("The list is empty\n");
        return;
    }
    node curr = head;
    while (curr) {
        printf("%d ", curr->data);
        curr = curr->next;
    }
    printf("\n");
}

```

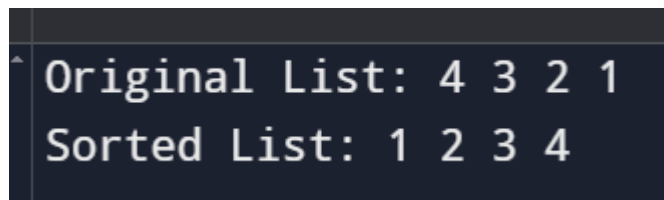
```

int main() {
    node head = NULL;
    head = insertAtHead(1, head);
    head = insertAtHead(2, head);
    head = insertAtHead(3, head);
    head = insertAtHead(4, head);
    // this will be inserted such as 4 3 2 1
    printf("Original List: ");
    display(head);

    head = bubbleSort(head);
    printf("Sorted List: ");
    display(head);

    return 0;
}

```



```

Original List: 4 3 2 1
Sorted List: 1 2 3 4

```

2.reverse

```

#include <stdio.h>
#include <stdlib.h>

struct n {
    int data;
    struct n *next;
};
typedef struct n *node;

node insertAtHead(int data, node head) {
    node temp = (node)malloc(sizeof(struct n));
    temp->data = data;
    temp->next = NULL;
    if (head == NULL)
        return temp;
}

```

```

    temp->next = head;
    head = temp;
    return head;
}

node reverse(node head) {
    node prev = NULL;
    node curr = head;
    while (curr) {
        node temp = curr->next;
        curr->next = prev;
        prev = curr;
        curr = temp;
    }
    head = prev;
    return head;
}

void display(node head) {
    if (head == NULL) {
        printf("The list is empty\n");
        return;
    }
    node curr = head;
    while (curr) {
        printf("%d ", curr->data);
        curr = curr->next;
    }
    printf("\n");
}

int main() {
    node head = NULL;
    head = insertAtHead(7, head);
    head = insertAtHead(3, head);
    head = insertAtHead(4, head);
    head = insertAtHead(5, head);

```

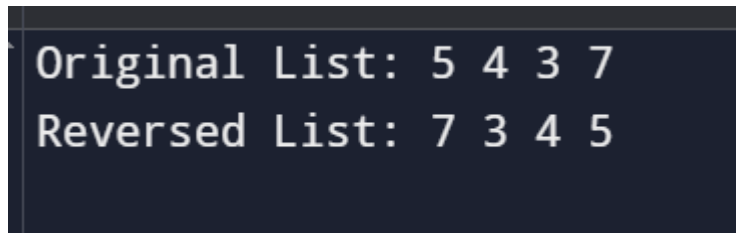
```

printf("Original List: ");
display(head);

head = reverse(head);
printf("Reversed List: ");
display(head);

return 0;
}

```



```

Original List: 5 4 3 7
Reversed List: 7 3 4 5

```

3.concatenate

```

#include <stdio.h>
#include <stdlib.h>

struct n {
    int data;
    struct n *next;
};
typedef struct n *node;

node insertAtHead(int data, node head) {
    node temp = (node)malloc(sizeof(struct n));
    temp->data = data;
    temp->next = NULL;
    if (head == NULL)
        return temp;
    temp->next = head;
    head = temp;
    return head;
}

node concatenate(node l1, node l2) {
    if (l1 == NULL) {
        return l2;
    }

```

```

    }
    if (l2 == NULL) {
        return l1;
    }
    node temp = l1;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = l2;
    return l1;
}

void display(node head) {
    if (head == NULL) {
        return;
    }
    node curr = head;
    while (curr) {
        printf("%d ", curr->data);
        curr = curr->next;
    }
    printf("\n");
}

int main() {
    node head1 = NULL, head2 = NULL;
    head1 = insertAtHead(1, head1);
    head1 = insertAtHead(2, head1);
    head1 = insertAtHead(3, head1);

    head2 = insertAtHead(8, head2);
    head2 = insertAtHead(9, head2);
    head2 = insertAtHead(10, head2);
    head1 = concatenate(head1, head2);
    display(head1);

    return 0;
}

```




4.stack

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct n {
    int data;
    struct n *next;
};
typedef struct n *node;
```

```
node top = NULL;
```

```
int isEmpty() {
    return top == NULL;
}
```

```
void push(int data) {
    if(top==NULL){
        top= (node)malloc(sizeof(struct n));
        top->data=data;
        top->next=NULL;
    }else{
        node temp = (node)malloc(sizeof(struct n));
        temp->data = data;
        temp->next = top;
        top = temp;
    }
}
```

```
void pop() {
    if (isEmpty()) {
        printf("The stack is empty\n");
        return;
    }
}
```

```
    node temp = top;
    top = top->next; // Update top to the next node
    free(temp);      // Free the popped node
}
```

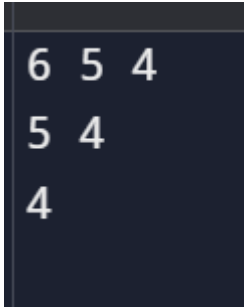
```
void display(node head) {
    if (head == NULL) {
        return;
    }
    node curr = head;
    while (curr) {
        printf("%d ", curr->data);
        curr = curr->next;
    }
    printf("\n");
}
```

```
int main() {
    push(4);
    push(5);
    push(6);
    display(top);

    pop();
    display(top);

    pop();
    display(top);

    return 0;
}
```



6 5 4
5 4
4

5.queue

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct n {  
    int data;  
    struct n *next;
```

```
};
```

```
typedef struct n *node;
```

```
node front = NULL, rear = NULL;
```

```
int isEmpty() {  
    return front == NULL;  
}
```

```
void enqueue(int data) {  
    node temp = (node)malloc(sizeof(struct n));  
    temp->data = data;  
    temp->next = NULL;
```

```
    if (isEmpty()) {  
        front = rear = temp;  
    } else {  
        rear->next = temp;  
        rear = temp;  
    }  
}
```

```
void dequeue() {  
    if (isEmpty()) {
```

```

        return;
    }
    node temp = front;
    front = front->next;
    if (front == NULL) {
        rear = NULL;
    }
    free(temp);
}

void display() {
    if (isEmpty()) {
        return;
    }
    node curr = front;
    while (curr) {
        printf("%d ", curr->data);
        curr = curr->next;
    }
    printf("\n");
}

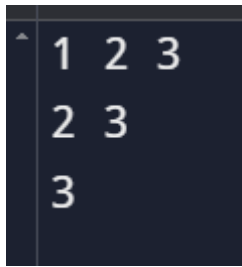
int main() {
    enqueue(1);
    enqueue(2);
    enqueue(3);
    display();

    dequeue();
    display();

    dequeue();
    display();

    return 0;
}

```



```
^ 1 2 3
  2 3
  3
```

LAB PROGRAM 7: DOUBLY LINKED LIST

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
    int data;
    struct node* next;
    struct node* prev;
};
```

```
typedef struct node* nod;
```

```
nod head = NULL;
```

```

nod createNode(int data) {
    nod temp = (nod)malloc(sizeof(struct node));
    temp->data = data;
    temp->next = NULL;
    temp->prev = NULL;
    return temp;
}

void insertAtEnd(int data) {
    nod temp = createNode(data);
    if (head == NULL) {
        head = temp;
    } else {
        nod ptr = head;
        while (ptr->next != NULL) {
            ptr = ptr->next;
        }
        ptr->next = temp;
        temp->prev = ptr;
    }
}

void deleteNode(int data) {
    nod temp = head;
    while (temp != NULL && temp->data != data) {
        temp = temp->next;
    }
    if (temp == NULL) {
        return;
    }
    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    } else {
        head = temp->next;
    }
    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }
}

```

```

    free(temp);
}

void insertLeft(int val, int data) {
    nod temp = head;
    while (temp != NULL && temp->data != data) {
        temp = temp->next;
    }
    if (temp == NULL) {
        return;
    }
    nod newTemp = createNode(val);
    newTemp->next = temp;
    newTemp->prev = temp->prev;
    if (temp->prev != NULL) {
        temp->prev->next = newTemp;
    } else {
        head = newTemp;
    }
    temp->prev = newTemp;
}

void display() {
    if (head == NULL) {
        return;
    }
    nod temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int ch, data, val;

    while(1) {

```

```

printf("Enter your choice\n ");
printf("1.add at last\n");
printf("2.add before value\n");
printf("3.delete the value\n");
printf("4. Display\n");

scanf("%d", &ch);

switch (ch) {
    case 1:
        printf("Enter data to insert at end: ");
        scanf("%d", &data);
        insertAtEnd(data);
        break;
    case 2:
        printf("Enter the value to insert: ");
        scanf("%d", &val);
        printf("Enter the data of the node before which to insert: ");
        scanf("%d", &data);
        insertLeft(val, data);
        break;
    case 3:

        printf("Enter data of node to delete: ");
        scanf("%d", &data);
        deleteNode(data);
        break;
    case 4:
        display();
        break;
    default:
        exit(0);
}
}

return 0;
}

```



```
Enter your choice
  1.add at last
  2.add before value
  3.delete the value
  4. Display
1
Enter data to insert at end: 12
Enter your choice
  1.add at last
  2.add before value
  3.delete the value
  4. Display
1
Enter data to insert at end: 14
Enter your choice
  1.add at last
  2.add before value
  3.delete the value
  4. Display
2
Enter the value to insert: 13 14
Enter the data of the node before which to insert: Enter your choice
  1.add at last
  2.add before value
  3.delete the value
  4. Display
```

```

Enter the value to insert: 13 14
Enter the data of the node before which to insert: Enter your choice
  1.add at last
  2.add before value
  3.delete the value
  4. Display
4
12 13 14
Enter your choice
  1.add at last
  2.add before value
  3.delete the value
  4. Display
3
Enter data of node to delete: 13
Enter your choice
  1.add at last
  2.add before value
  3.delete the value
  4. Display
4
12 14
Enter your choice
  1.add at last
  2.add before value

```

LAB PROGRAM 8:

CODE: BINARY SEARCH TREE

// Online C compiler to run C program online

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct n{
```

```
    int data;
```

```
    struct n *left,*right;
```

```

};
typedef struct n *node;
node createNode(int data){
    node temp=(node)malloc(sizeof(struct n));
    temp->data=data;
    temp->left=temp->right=NULL;
    return temp;
}

node insert(node root,int data){
    if(root==NULL){
        return createNode(data);
    }
    else if(data<root->data){
        root->left=insert(root->left,data);
    }else if(data>root->data){
        root->right=insert(root->right,data);
    }
    return root;
}

void preorder(node root){
    if(root==NULL){
        return ;
    }
    printf("%d ",root->data);
    preorder(root->left);
    preorder(root->right);
}

void postorder(node root){
    if(root==NULL){
        return ;
    }
    postorder(root->left);
    postorder(root->right);
    printf("%d ",root->data);
}

void inorder(node root){

```

```

if(root==NULL){
    return ;
}
inorder(root->left);
printf("%d ",root->data);
inorder(root->right);
}

```

```

int main(){
    node root=NULL;
    int ch;
    int data;
    do{
        printf("enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit");
        printf("\n");
        scanf("%d",&ch);
        switch(ch){
            case 1:
                printf("enter the data to put in the tree\n");
                scanf("%d",&data);
                root=insert(root,data);
                break;

            case 2:
                preorder(root);
                printf("\n");
                break;

            case 3:
                postorder(root);
                printf("\n");
                break;

            case 4:
                inorder(root);

```

```
    printf("\n");  
    break;  
  
    case 5:  
        exit(0);  
        break;  
}
```

```
}while(1);  
return 0;  
}
```

OUTPUT:

```
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
1
enter the data to put in the tree
1
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
1
enter the data to put in the tree
2
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
1
enter the data to put in the tree
3
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
1
enter the data to put in the tree
4
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
1
enter the data to put in the tree
5
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
1
enter the data to put in the tree
6
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
1
```

```

enter the data to put in the tree
5
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
1
enter the data to put in the tree
6
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
1
enter the data to put in the tree
7
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
4
1 2 3 4 5 6 7
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
2
1 2 3 4 5 6 7
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
3
7 6 5 4 3 2 1
enter the choice 1.insert 2.preorder 3.postorder 4.inorder 5.exit
5

=== Code Execution Successful ===

```

LAB PROGRAM 9:

BFS IN A GRAPH

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#include <stdlib.h>
```

```
int queue[100];
```

```
int front = -1, rear = -1;
```

```
int isEmpty() {
```

```

    return front == -1;
}

void push(int value) {
    if (rear == 99) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = value;
}

int pop() {
    if (isEmpty()) {
        printf("Queue Underflow\n");
        return -1;
    }
    int data = queue[front];
    if (front == rear) {
        front = rear = -1;
    } else {
        front++;
    }
    return data;
}

```

```

void bfs(int graph[][100], int visited[], int n, int start) {

    visited[start] = 1;
    push(start);

    while (!isEmpty()) {
        int node = pop();
        printf("%d ", node);

        for (int i = 1; i <= n; i++) {
            if (graph[node][i] && !visited[i]) {

```



```

        visited[i] = 1;
        push(i);
    }
}
}

int main() {
    int n;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    int graph[100][100] = {0};
    int *visited = (int *)calloc(n + 1, sizeof(int));

    printf("Enter the adjacency matrix:\n");
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    bfs(graph, visited, n, 1);

    return 0;
}
output:

```

```
Enter the number of vertices: 5
Enter the adjacency matrix:
0 1 1 0 0
1 0 0 1 1
1 0 0 0 0
0 1 0 0 0
0 1 0 0 0
1 2 3 4 5
```

DFS OF A GRAPH:

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#include<stdlib.h>
```

```
void dfs(int graph[][100], int visited[], int n,int start) {
    visited[start] = 1;
    printf("%d ", start);
    for (int i = 1; i < n+1; i++) {
        if (graph[start][i] && !visited[i]) {
            dfs(graph, visited,n,i);
        }
    }
}
```

```
int main() {
    int n;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    int graph[n+1][n+1];
    int *visited=(int *)calloc(n+1,sizeof(int));
```

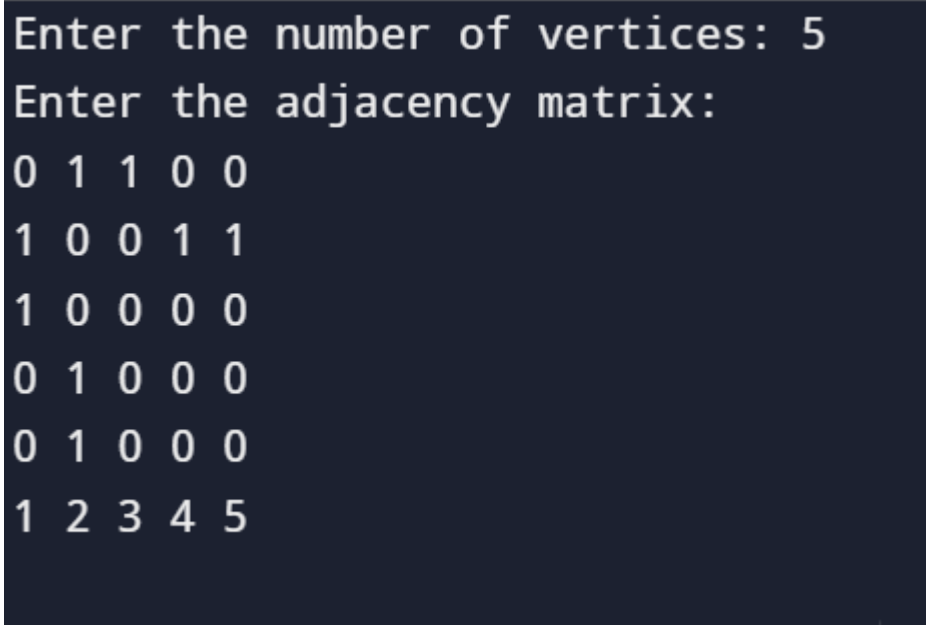
```

printf("now enter the adjacency matrix:\n");
for (int i = 1; i < n+1; i++) {
    for (int j = 1; j < n+1; j++) {
        scanf("%d", &graph[i][j]);
    }
}

dfs(graph, visited, n,1);

return 0;
}

```



```

Enter the number of vertices: 5
Enter the adjacency matrix:
0 1 1 0 0
1 0 0 1 1
1 0 0 0 0
0 1 0 0 0
0 1 0 0 0
1 2 3 4 5

```

LAB 10: EMPLOYEE RECORDS(HASHING)

```

#include <stdio.h>
#include <stdlib.h>

```

```

int key[20], n, m;
int *ht, index;
int count = 0;

```

```

void insert(int key) {
    index = key % m;
    while (ht[index] != -1) {

```

```

        index = (index + 1) % m;
    }
    ht[index] = key;
    count++;
}

void display() {
    int i;
    if (count == 0) {
        printf("\nHash Table is empty");
        return;
    }
    printf("\nHash Table contents are:\n");
    for (i = 0; i < m; i++)
        printf("\nT[%d] --> %d", i, ht[i]);
}

void main() {
    int i;

    printf("\nEnter the number of employee records (N): ");
    scanf("%d", &n);

    printf("\nEnter the two-digit memory locations (m) for hash table: ");
    scanf("%d", &m);

    ht = (int *)malloc(m * sizeof(int));
    for (i = 0; i < m; i++)
        ht[i] = -1;

    printf("\nEnter the four-digit key values (K) for N Employee Records:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &key[i]);

    for (i = 0; i < n; i++) {
        if (count == m) {
            printf("\nHash table is full. Cannot insert the record %d key", i + 1);
            break;

```

```
    }  
    insert(key[i]);  
}  
  
display();  
}
```

```
Enter the number of employee records (N): 5
```

```
Enter the two-digit memory locations (m) for hash table: 7
```

```
Enter the four-digit key values (K) for N Employee Records:
```

```
1234
```

```
5678
```

```
9101
```

```
1121
```

```
3141
```

```
Hash Table contents are:
```

```
T[0] --> -1
```

```
T[1] --> 5678
```

```
T[2] --> 1234
```

```
T[3] --> 9101
```

```
T[4] --> 1121
```

```
T[5] --> 3141
```

```
T[6] --> -1
```

```
=== Code Exited With Errors ===
```

